

INTERNSHIP PROJECT REPORT

TASKFLOW

A Full-Stack Task Management and Team Collaboration Platform

Submitted By:

Emaan Abid

Internship Organization:

FORTEX

Internship Project:

2026

Table of Contents

1. Project Overview	7
2. Problem Statement	7
3. Project Objectives	8
4. Scope of the Project.....	8
User Management	9
Project Management.....	9
Task Management	9
Collaboration	10
Analytics.....	10
Account and System Management.....	10
5. User Authentication	10
Signup.....	11
Login	11
Logout.....	11
Password Reset	11
6. Dashboard	11
Dashboard functionality	11
Team Members.....	12
Recent Activity.....	12
Project Progress.....	12
Deadlines	12
Task Completion Chart.....	12
Dashboard Search.....	12
7. Project Management	12
Project Creation.....	13
Project Updating	13
Project Deletion.....	13
Project Permissions	13
8. Task Management.....	13
Task CRUD.....	14

Task Assignment	14
Task Permissions	14
Project Owner	14
Assignee.....	14
9. Kanban Board	14
10. Drag-and-Drop Task Management	15
11. Task Creation.....	15
Floating Add Button.....	15
Column Add Buttons	15
12. Task Drawer	15
Save Task Loading State.....	16
Task Comments.....	16
Task Attachments	16
Task Activity Timeline	16
13. Activity Page.....	16
14. Search Functionality	17
15. Team Management	17
16. Invitation System.....	17
17. Notifications.....	17
Notification Features	17
18. User Profile Management	18
19. Account Management.....	18
20. Notification Management	18
21. Billing Management	18
Billing Features.....	19
Payment Method	19
22. Recycle Bin / Soft Delete.....	19
23. Backend Architecture.....	20
Middleware Layer	20
Controller Layer.....	20
Service Layer	20

Repository Layer	21
Global Error Handling	21
24. Error Handling	21
25. Database	21
Automated Database Setup	22
26. Redis Caching	22
27. API Documentation	23
28. Environment Configuration	23
29. Security	23
JWT Authentication	23
Role-Based Authorization	24
Password Management	24
Two-Factor Authentication	24
Secure Payment Metadata	24
Backend Validation	24
30. Frontend Technology Stack	24
31. Backend Technology Stack	25
32. System Architecture	25
33. Complete Application Workflow	26
34. Functional Requirements	26
35. Non-Functional Requirements	27
36. Responsive Design	27
37. External/Supporting Services	27
38. Installation Requirements	27
Recommended hardware	27
Supported OS	28
39. Local Installation	28
Step 1: Clone Repository	28
Step 2: Backend Setup	28
Step 3: Frontend Setup	28
Step 4: Running the Complete Application	29

	Step 5: API Documentation	29
40.	Deployment	30
41.	Live Application	30
42.	Testing and Validation	30
43.	Strengths of the System	32
	1. Centralized Management	32
	2. Visual Workflow	32
	3. Role-Based Access	32
	4. Collaboration	32
	5. Analytics	32
	6. Performance	32
	7. Maintainable Backend	32
	8. Production Deployment	32
	Limitations	32
44.	Future Improvements	33
	Email Invitations	33
	Advanced Analytics	33
	Calendar Integration	33
	Advanced Search	33
45.	Conclusion	34

Table of Figures:

Figure 1: The complete system architecture of the TaskFlow	25
Figure 2: User workflow of the TaskFlow.....	26

1. Project Overview

TaskFlow is a modern full-stack task management and team collaboration web application designed to help individuals and teams organize projects, manage tasks, monitor progress, and collaborate efficiently through a centralized platform.

The application provides a Kanban-style project management workflow where users can create projects, manage tasks, assign tasks to team members, update task statuses using drag-and-drop functionality, communicate through comments, attach files, monitor activities, receive notifications, and analyze project progress.

TaskFlow also provides account, profile, notification, billing, invitation, and activity-management functionality, making it more than a simple task manager. The application is designed as a complete project-management platform with role-based permissions, real-time-oriented dashboard updates, caching, authentication, automated database setup, and production deployment.

The system consists of two major components:

- **Frontend:** React-based user interface
- **Backend:** Node.js/Express REST API connected to PostgreSQL through Prisma ORM

The frontend and backend are deployed separately, with the frontend hosted on Vercel and the backend and the database hosted on Railway.

2. Problem Statement

Managing projects and tasks through scattered tools can make it difficult for teams to maintain a clear understanding of project progress, task ownership, deadlines, communication, and recent activities.

Common problems include:

- Difficulty organizing multiple projects
- Lack of centralized task management
- Poor visibility of task ownership
- Difficulty tracking project progress
- Lack of clear task statuses
- Communication being separated from tasks
- Difficulty tracking project activity
- Lack of centralized notifications
- Difficulty managing team members and invitations
- No clear role-based control over projects and tasks

- Difficulty monitoring deadlines

TaskFlow addresses these problems by providing a centralized platform that integrates projects, tasks, users, communication, notifications, activities, and account management into a single application.

3. Project Objectives

The main objectives of TaskFlow are:

1. To provide a centralized task and project management platform.
2. To allow users to create and manage multiple projects.
3. To provide a Kanban-based workflow for task management.
4. To allow project owners to assign tasks to team members.
5. To implement role-based permissions for projects and tasks.
6. To provide drag-and-drop task management.
7. To provide task comments and attachments.
8. To maintain activity history for projects and tasks.
9. To provide dashboard analytics and project statistics.
10. To provide centralized in-app notifications.
11. To provide user profile and account management.
12. To provide billing and subscription management.
13. To provide invitation functionality for collaboration.
14. To provide search functionality for tasks and dashboard information.
15. To implement secure authentication and authorization.
16. To provide automated database setup.
17. To standardize backend error handling.
18. To organize backend code using layered architecture.
19. To improve application performance using caching.
20. To deploy the complete application to production.

4. Scope of the Project

TaskFlow covers the following major areas:

User Management

- Signup
- Login
- Logout
- Password reset
- Profile management
- Profile image management
- Email management
- Password management
- Two-factor authentication
- Account deletion

Project Management

- Project creation
- Project viewing
- Project updating
- Project deletion
- Project progress tracking
- Project members
- Project permissions
- Soft deletion

Task Management

- Task creation
- Task viewing
- Task updating
- Task deletion
- Task assignment
- Task comments
- Task attachments
- Task activity timeline
- Task search

- Task status management
- Drag-and-drop

Collaboration

- Team members
- Invitations
- Comments
- Attachments
- Activity tracking
- Notifications

Analytics

- Project statistics
- Task statistics
- Completed task statistics
- Deadlines
- Project progress
- Task completion chart
- Recent activities

Account and System Management

- Notification preferences
- Billing
- Subscription plans
- Invoice information
- API documentation
- Automated database setup
- Error handling
- Caching

5. User Authentication

TaskFlow provides authentication functionality for protecting user accounts and application resources.

Signup

New users can create accounts through the signup page.

The authentication workflow includes:

User → Signup Form → Backend API → Validation → User Creation → Database → Account Created.

Login

Registered users can log into the application using their credentials.

The application uses JWT-based authentication for authenticated API requests.

The general workflow is:

Login → Credentials Verification → JWT Generation → Token Stored by Frontend → Authenticated API Requests.

The frontend sends the authentication token with protected API requests.

Logout

Logout functionality has been implemented on both:

- Frontend
- Backend

This ensures that logout is not only a client-side operation but is also handled by the backend authentication system.

Password Reset

The application provides a Forgot Password feature.

Users can initiate password recovery from the login page and reset their password.

6. Dashboard

The Dashboard acts as the central overview of the user's TaskFlow workspace.

It provides real-time-oriented information about projects, tasks, team members, deadlines, progress, and activities.

Dashboard functionality

The dashboard displays:

- Total projects
- Task statistics
- Completed tasks

- Team members
- Recent activities
- Project progress
- Upcoming deadlines
- Task completion chart
- Dashboard search

Team Members

The dashboard displays team members associated with the user's projects.

Users can click a team member:

- Name
- Profile image

to open that member's profile in read-only mode.

Recent Activity

The dashboard displays recent activity related to the user's projects and tasks.

This allows users to quickly identify recent changes without manually opening every project.

Project Progress

Project progress is represented using the number of completed tasks compared to the total number of tasks.

Deadlines

Upcoming task deadlines are displayed on the dashboard. The deadline section uses a scrollable view so that multiple upcoming deadlines can be accessed without making the dashboard excessively long.

Task Completion Chart

The dashboard includes a task completion chart that displays the number of tasks completed on a particular day. This gives users a visual understanding of productivity and task completion trends.

Dashboard Search

A search bar has been implemented on the dashboard to make information easier to find.

7. Project Management

TaskFlow provides complete project CRUD functionality.

CRUD stands for:

- Create
- Read
- Update
- Delete

Users can create and manage projects through the Projects section.

Project Creation

Users can create projects by providing the required project information.

After creation, the project becomes available in the user's project workspace.

Project Updating

Project owners have permission to update project information.

Project Deletion

Project owners have permission to delete projects. TaskFlow also implements soft deletion for projects. Instead of immediately removing the project from the database, it can be moved into a deleted state, allowing the system to maintain the record and support recovery-related functionality.

Project Permissions

Project ownership determines project-level control.

The project owner has full control over project management operations.

This includes:

- Updating the project
- Deleting the project
- Managing task assignments

8. Task Management

TaskFlow provides complete task management inside projects.

Each task can contain information such as:

- Title
- Description
- Status
- Due date
- Assignee

- Comments
- Attachments

Task CRUD

Users can:

- Create tasks
- View tasks
- Update tasks
- Delete tasks

Task Assignment

The project owner can assign tasks to project members. The project owner can also assign a task to themselves. This provides flexibility for both team-based and self-managed tasks.

Task Permissions

Task permissions are role-based.

Project Owner

The project owner has full control over tasks.

The owner can:

- Create tasks
- Update tasks
- Delete tasks
- Assign tasks
- Manage task information

Assignee

An assignee can modify tasks assigned to them according to the implemented permissions.

This prevents unauthorized users from modifying tasks that they do not have permission to manage.

9. Kanban Board

TaskFlow uses a Kanban-style workflow for organizing tasks.

The available task states are:

TO DO, IN PROGRESS, REVIEW, COMPLETE.

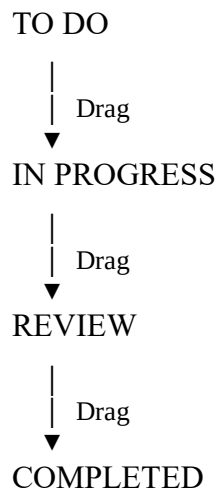
The Kanban board allows users to understand the current state of every task visually.

10. Drag-and-Drop Task Management

TaskFlow implements drag-and-drop functionality using DnD Kit.

Users can drag a task from one status column to another.

For example:



11. Task Creation

Tasks can be created from multiple locations within the Kanban board.

Users can create tasks through:

Floating Add Button

A floating plus button is available at the bottom of the Kanban board.

Column Add Buttons

Each status column also contains a plus button.

12. Task Drawer

Task information is displayed inside a task drawer.

The drawer provides access to:

- Task information
- Comments
- Attachments
- Activity timeline
- Task editing
- Task saving

Save Task Loading State

The Save Task button has been improved so that after the user clicks it:

Saving...

is displayed and the button becomes disabled while the operation is in progress.

This prevents:

- Duplicate submissions
- Multiple API requests
- Accidental repeated updates

Task Comments

Users can communicate directly within tasks using comments. Comments provide task-specific communication rather than requiring users to switch to another communication platform.

Task Attachments

TaskFlow provides attachment functionality inside tasks. Users can attach files to tasks and access those attachments through the task interface.

Task Activity Timeline

Each task contains an activity timeline showing relevant task activities. This allows users to understand the history of changes made to a task.

13. Activity Page

TaskFlow provides a dedicated Activity page. The Activity page includes frontend and backend implementation. Activity records are displayed using pagination. Instead of loading all activities at once:

Page 1 → Latest 20 activities

Page 2 → Next 20 activities

...

This improves usability and reduces unnecessary data loading. Activities can also navigate users to their respective project/task context.

14. Search Functionality

TaskFlow provides search functionality for tasks in the Projects page. Users can search for tasks instead of manually scanning through all Kanban columns. Dashboard search functionality has also been implemented.

15. Team Management

TaskFlow provides team collaboration functionality.

Users can work with team members by:

- Adding members to projects
- Assigning tasks
- Viewing member profiles
- Inviting members
- Collaborating through comments
- Tracking member-related activity

16. Invitation System

TaskFlow includes an invitation system for bringing users into projects. The implemented invitation functionality supports the application's in-app collaboration workflow. The invitation system has been developed as part of the team/project collaboration features.

17. Notifications

TaskFlow includes an in-app notification system.

Notifications are generated for relevant updates made to projects and tasks owned by the user.

Notification Features

Users can:

- View notifications
- See unread notification count
- Mark all notifications as read

- Delete notifications

This allows users to immediately know when new activity requires their attention.

18. User Profile Management

The Settings section provides profile-management functionality.

Users can:

- Update profile information
- View profile information
- Change profile image
- Delete profile image

Other users can view profiles in read-only mode where applicable.

19. Account Management

TaskFlow provides account-level controls.

Users can update their account email. Users can change their account password. Users can enable 2FA for additional account security. Users can permanently delete their account.

20. Notification Management

Users can control their notification preferences.

Notification settings include categories related to:

- Activity
- Projects
- System notifications

Users can control notification delivery through:

- Email
- Push notifications

21. Billing Management

TaskFlow includes billing and subscription management. The Billing section provides information about the user's current subscription.

The system supports plans including:

Plan	Monthly Price
FREE	\$0
PRO	\$12
ENTERPRISE	\$39

Billing Features

The billing system provides:

- Current plan
- Plan selection
- Plan updates
- Renewal information
- Payment method management
- Invoice history
- Invoice status
- Billing cycle

Payment Method

The system stores only safe payment metadata such as:

- Card Brand
- Last 4 Digits
- Expiry

Sensitive information such as:

- Full Card Number
- CVV

is not stored by the TaskFlow backend.

22. Recycle Bin / Soft Delete

TaskFlow implements soft deletion for projects.

Instead of immediately deleting a project from the database, the system maintains its record in a deleted state.

This approach provides safer deletion behavior and allows deleted data to be handled separately.

23. Backend Architecture

The backend is implemented using a layered architecture.

The architecture separates responsibilities between different layers.

HTTP Request → Routes → Middleware → Controller → Service → Repository → PostgreSQL

Middleware Layer

The middleware layer handles cross-cutting functionality such as:

- Authentication
- Request validation
- Error handling
- Other request-level processing

Controller Layer

Controllers are responsible for:

- Receiving HTTP requests
- Extracting request information
- Calling services
- Returning HTTP responses

Controllers do not contain the application's main business logic.

Service Layer

The service layer contains business logic.

For example, billing operations such as:

- Plan management
- Pricing
- Renewal calculations
- Payment metadata updates
- Invoice creation

are handled by the service layer.

Repository Layer

The repository layer handles database interaction.

This separation prevents database queries from being unnecessarily mixed with controllers and business logic.

Global Error Handling

Unhandled errors are passed to a centralized error-handling middleware.

This provides standardized API error responses.

24. Error Handling

TaskFlow includes standardized backend error handling.

The purpose of centralized error handling is to provide consistent responses for:

- Validation errors
- Authentication errors
- Authorization errors
- Not-found errors
- Database errors
- Unexpected server errors

Instead of every controller handling errors independently, errors can be mapped and passed to the global error handler.

25. Database

TaskFlow uses:

- PostgreSQL
- Prisma ORM

The application database stores information related to:

- Users
- Projects
- Tasks
- Team/project memberships
- Team Invitation
- Comments

- Attachments
- Activities
- Notifications
- Notification Settings
- Billing
- Invoices
- Token Blacklist
- Other application entities

The project uses CUID string identifiers for entities.

Automated Database Setup

A database setup script has been implemented to simplify installation.

Developers can execute:

```
npm run setup
```

The setup process automatically handles:

Generate Prisma Client → Database setup → Schema synchronization → Application ready.

This reduces the need for new developers to manually execute multiple Prisma commands.

26. Redis Caching

Redis caching has been implemented to improve performance for frequently accessed data.

Caching has been applied to areas including:

- Dashboard
- Projects
- Tasks

The general architecture is:

Frontend → Backend API → Check Redis Cache

If Cache Hit → Return Cached Data.

If Cache Miss → PostgreSQL → Store in Redis → Return Data.

Caching reduces unnecessary database queries for frequently accessed information.

27. API Documentation

TaskFlow provides Swagger API documentation.

Swagger is available at: <http://localhost:5000/api-docs>

The API documentation includes:

- Endpoint descriptions
- Request information
- Response examples
- Parameter explanations
- Possible error responses

This makes the backend easier to understand, test, and integrate.

28. Environment Configuration

TaskFlow provides “.env.example” files so that developers can understand which environment variables are required.

The basic workflow is:

```
cp .env.example .env
```

Windows Command Prompt:

```
copy .env.example .env
```

Windows PowerShell:

```
Copy-Item .env.example .env
```

The developer then updates the environment variables according to their local setup.

29. Security

Security considerations implemented in TaskFlow include:

JWT Authentication

Protected resources require authenticated requests.

Role-Based Authorization

Project owners and assignees have different permissions.

Password Management

Users can change and reset passwords.

Two-Factor Authentication

Users can enable 2FA.

Secure Payment Metadata

The application does not store:

- Full card number
- CVV

Only non-sensitive card metadata is stored.

Backend Validation

Request validation and centralized error handling are used to prevent invalid operations from being processed silently.

30. Frontend Technology Stack

The frontend uses:

Technology	Purpose
React	UI development
TypeScript	Type safety
Vite	Development/build tooling
Tailwind CSS	Styling
React Router	Client-side routing
Axios	API communication
DnD Kit	Drag-and-drop
Recharts	Charts and analytics
Lucide React / React Icons	UI icons

31. Backend Technology Stack

The backend uses:

Technology	Purpose
Node.js	Runtime
Express.js	REST API
TypeScript	Type safety
Prisma	ORM/database access
PostgreSQL	Relational database
JWT	Authentication
Swagger	API documentation
Redis	Caching

32. System Architecture

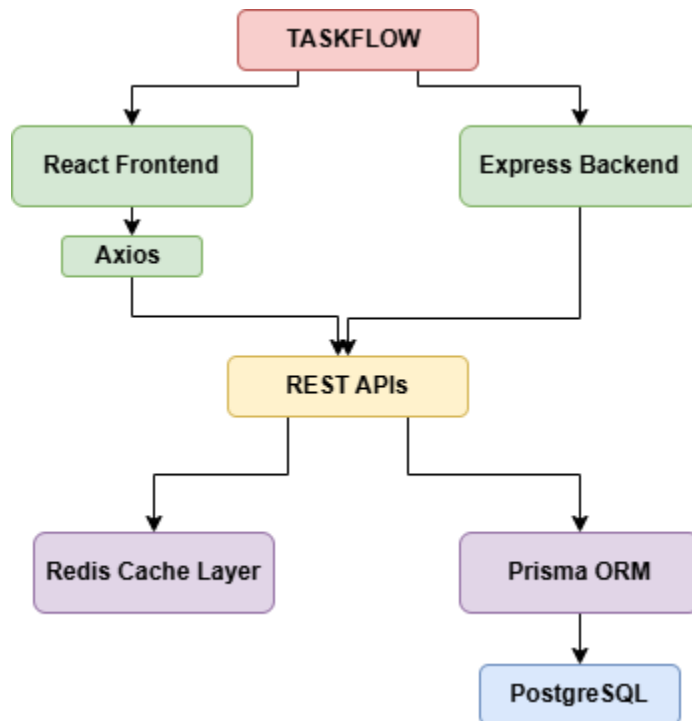


Figure 1: The complete system architecture of the TaskFlow

33. Complete Application Workflow

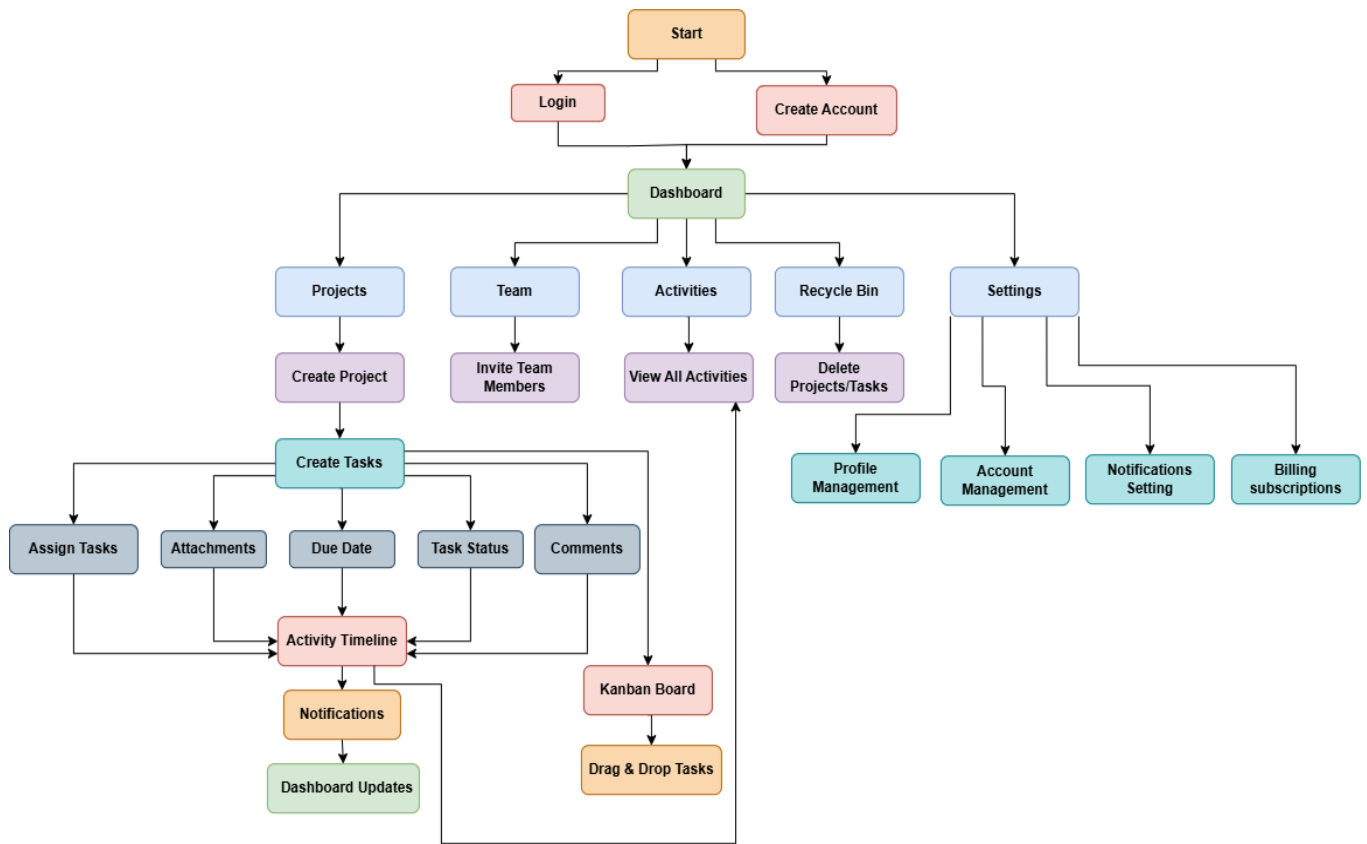


Figure 2: User workflow of the TaskFlow

34. Functional Requirements

- **Authentication:** Offers complete security workflows including Signup, Login, Logout, Password Reset, JWT Authentication, and Two-Factor Authentication (2FA).
- **Projects:** Supports full lifecycle management with Create, View, Update, Delete, Soft Delete, Project Permissions, and Progress tracking.
- **Tasks:** Provides detailed task handling with Create, View, Update, Delete, Assign Task, Self Assignment, Comments, Attachments, Activity Timeline, Search, Drag & Drop, and Task Statuses.
- **Dashboard:** Delivers high-level operational insights via Project Statistics, Task Statistics, Team Members overview, Recent Activity feeds, Deadlines, Progress metrics, Completion Charts, and Search.
- **Account:** Enables user profile control through Profile Management, Profile Image uploads, Email Updates, Password Changes, 2FA settings, and Account Deletion.
- **Notifications:** Keeps users updated using In-App Notifications, Unread Count indicators, Mark All as Read options, and Delete Notifications.

- **Billing:** Handles subscription workflows including Current Plan details, Plan Selection, Payment Method management, Invoice History, Renewal Information, and Subscription Plans.

35. Non-Functional Requirements

- **Performance:** Uses Redis caching for high-frequency dashboard, project, and task data, along with pagination for activity logs to streamline data retrieval.
- **Scalability:** Enforces a clean separation between Controllers, Services, Repositories, and the Database to simplify backend expansion and lower primary database load.
- **Maintainability:** Built with TypeScript, a layered backend architecture, reusable React components, Service/API separation, centralized error handling, and environment configuration.
- **Usability:** Enhances user experience through a Kanban workflow, drag-and-drop mechanics, responsive UI, search capabilities, dashboard analytics, notifications, a task drawer, and profile management.

36. Responsive Design

TaskFlow provides a responsive user interface so that the application's interface can adapt to different screen sizes.

37. External/Supporting Services

The project also includes integration with **Bee Keeper**, as part of the implemented project work.

The exact role of this integration should be documented separately according to the final deployed configuration.

38. Installation Requirements

Before running TaskFlow locally, the following software is required:

- Node.js v18
- npm
- PostgreSQL v14

Recommended hardware

- Minimum 4 GB RAM
- At least 1 GB free storage
- Modern dual-core processor

Supported OS

- Windows 10/11
- Linux
- MacOS

39. Local Installation

Step 1: Clone Repository

git clone <https://github.com/EmaanAbid2711/TaskFlow-Frontend.git>

Navigate into the project: `cd TaskFlow-Frontend`

Project structure:

TaskFlow-Frontend

```
├── taskflow
│   ├── frontend
│   └── backend
```

Step 2: Backend Setup

Navigate to: `cd taskflow/backend`

Install dependencies:

`npm install`

Create environment file.

For Linux/macOS: `cp .env.example .env`

For Windows CMD: `copy .env.example .env`

For Windows PowerShell: `Copy-Item .env.example .env`

Configure the environment variables.

Create a PostgreSQL database: `taskflow_db`, or use another database name and update the `DATABASE_URL`

Run: `npm run setup`

Then: `npm run dev`

The backend will run on `http://localhost:5000`

Step 3: Frontend Setup

Open another terminal.

Navigate to: `cd taskflow/frontend`

Install dependencies:

`npm install`

Create the environment file: `cp .env.example .env`

On Windows CMD: `copy .env.example .env`

On PowerShell: `Copy-Item .env.example .env`

Configure the frontend environment variables.

Start the frontend: `npm run dev`

Frontend will run on: `http://localhost:5173`

Step 4: Running the Complete Application

Two terminals are required.

Terminal 1 — Backend

`cd taskflow/backend`

`npm install`

`npm run setup`

`npm run dev`

Terminal 2 — Frontend

`cd taskflow/frontend`

`npm install`

`npm run dev`

Open: `http://localhost:5173`

Step 5: API Documentation

After starting the backend, Swagger documentation is available at: `http://localhost:5000/api-docs`

Swagger provides an interface for:

- Viewing endpoints
- Understanding request parameters
- Viewing request bodies
- Viewing response structures
- Testing APIs
- Understanding possible errors

40. Deployment

TaskFlow is deployed as a full-stack production application.

Frontend is hosted on Vercel. The backend is hosted on Railway, and the database is hosted on Railway PostgreSQL.

The frontend communicates directly with the deployed backend.

41. Live Application

Based on the deployment information provided:

Service	Platform	URL
Frontend	Vercel	https://task-flow-frontend-woad.vercel.app
Backend	Railway	https://taskflow-backend-production-df3c.up.railway.app
API Documentation	Railway	https://taskflow-backend-production-df3c.up.railway.app/api-docs

42. Testing and Validation

TaskFlow functionality has been validated through development and production testing.

Important areas tested include:

Authentication

- Signup
- Login
- Logout
- Password reset
- Protected requests

Projects

- Project creation
- Project modification
- Project deletion
- Owner permissions
- Soft deletion

Tasks

- Task creation

- Task modification
- Task deletion
- Task assignment
- Self assignment
- Status changes
- Drag-and-drop
- Comments
- Attachments

Dashboard

- Project statistics
- Task statistics
- Progress
- Deadlines
- Recent activities
- Completion chart
- Search

Activity

- Activity creation
- Activity retrieval
- Pagination
- Navigation

Notifications

- Notification creation
- Unread count
- Mark all as read
- Delete notification

Billing

- Current plan
- Plan selection
- Payment metadata

- Billing information
- Invoice information

Deployment

- Frontend-to-backend communication
- Production API requests
- Database connectivity
- Swagger availability

43. Strengths of the System

TaskFlow provides several advantages over a basic task-management application.

1. Centralized Management

Projects, tasks, team members, comments, files, activities, and notifications are available from one platform.

2. Visual Workflow

The Kanban board provides an easy-to-understand representation of task progress.

3. Role-Based Access

Project owners and task assignees have different permissions.

4. Collaboration

Comments, attachments, invitations, activities, and notifications support teamwork.

5. Analytics

Dashboard statistics and charts provide an overview of productivity and project progress.

6. Performance

Redis caching and activity pagination reduce unnecessary data retrieval.

7. Maintainable Backend

The layered backend architecture separates responsibilities.

8. Production Deployment

The application is deployed and accessible through a production frontend and backend.

Limitations

Based on the currently provided implementation, some areas can still be extended in future versions.

The current system can be improved with:

- Advanced project templates
- Calendar integrations
- More sophisticated billing/payment processing

44. Future Improvements

The following features can be considered for future versions:

Email Invitations

Although invitation functionality has been implemented, the email-based invitation workflow can be further expanded and refined if required.

Advanced Analytics

Future versions can provide:

- Weekly productivity reports
- Monthly productivity reports
- Team performance reports
- Project velocity

Calendar Integration

Tasks could be synchronized with external calendars.

Advanced Search

Search could be expanded to include:

- Projects
- Users
- Comments
- Activities
- Attachments

Real-Time Collaboration

WebSockets could be introduced for more immediate updates across users.

Additional Integrations

TaskFlow could integrate with external tools such as:

- GitHub

- Slack
- Google Calendar
- Microsoft Teams

45. Conclusion

TaskFlow is a complete full-stack task and project management platform developed to simplify project organization, task tracking, team collaboration, and productivity monitoring.

The application combines:

- Secure authentication
- Project management
- Task management
- Kanban workflow
- Drag-and-drop
- Task assignment
- Comments
- Attachments
- Activity tracking
- Dashboard analytics
- Notifications
- Invitations
- Profile management
- Account management
- Billing
- Role-based permissions
- Soft deletion
- Redis caching
- Automated database setup
- Standardized error handling
- Swagger documentation
- Production deployment

The system follows a structured architecture where the React frontend communicates with an Express/TypeScript backend through REST APIs. Prisma provides database access to PostgreSQL, while Redis is used to improve performance for frequently accessed information.

The completed system demonstrates the practical application of frontend development, backend development, database management, authentication, authorization, API design, caching, software architecture, responsive UI design, deployment, and collaborative project management.

TaskFlow is therefore not only a task-management interface but a complete full-stack software engineering project demonstrating the development and deployment of a production-oriented web application.